



SOFTWARE DEV

MERN STACK DEVELOPMENT REPORT

**Node.js Backend Basics: Modules, Core
Modules, and File System Operations**

Muneeb Ur Rehman

<https://github.com/955muneeb/Glaxit-internship/tree/main/week-7/backend-basics>

1. Introduction

The purpose of this lab was to understand the fundamental concepts of **Node.js backend development**, particularly Node.js modules, the Event Loop, CommonJS and ES Modules, core modules, and asynchronous file handling. Practical implementation was performed using Node.js to read, modify, and write data stored in a JSON file.

3. Tools and Technologies Used

- **Node.js**
 - **JavaScript**
 - **Visual Studio Code**
 - **JSON**
 - Node.js Core Modules:
 - `fs/promises`
 - `path`
 - `http`
-

4. Theory

4.1 Node.js Event Loop

Node.js uses an **Event Loop** to handle multiple operations efficiently. Although JavaScript execution runs on a main thread, Node.js does not unnecessarily wait for slow I/O operations such as file reading or database requests.

For example, when Node.js starts reading a large file, it can continue executing other code while the file operation is being completed. Once the operation finishes, its result is handled by Node.js.

This approach is known as **non-blocking I/O** and allows Node.js applications to efficiently handle many requests.

4.2 CommonJS and ES Modules

Node.js supports different module systems.

CommonJS:

```
const fs = require("fs");
```

It uses:

```
require()  
module.exports
```

ES Modules:

```
import fs from "fs";
```

It uses:

```
import
```

`export`

In this lab, **ES Modules** were used through the following `package.json` configuration:

```
"type": "module"
```

5. Node.js Core Modules

Node.js provides several built-in modules that can be used without installing external packages.

fs

The File System module is used to:

- Read files
- Write files
- Create files
- Update files
- Delete files

path

The `path` module helps work with file and directory paths in a reliable way across different operating systems.

http

The `http` module provides functionality for creating basic web servers and handling HTTP requests and responses.

6. Understanding `package.json`

Field	Purpose
<code>name</code>	Defines the project name
<code>version</code>	Defines the project version
<code>description</code>	Provides information about the project
<code>main</code>	Defines the main entry file
<code>scripts</code>	Contains commands that can be executed with <code>npm</code>
<code>author</code>	Specifies the project author
<code>license</code>	Defines the project license
<code>type</code>	Specifies the module system

7. Practical Implementation

A JSON file named `users.json` was used to store user information.

Example initial data:

```
[
  {
    "id": 1,
    "name": "Ali",
    "role": "Admin"
  },
  {
    "id": 2,
    "name": "Ahmed",
    "role": "User"
  }
]
```

The Node.js program was then created to read this file, add a new user, and save the updated information.

Main implementation

```
import fs from 'fs/promises';
import path from 'path';

const filePath = path.resolve('users.json');

async function manageUsers() {
  try {
    console.log("1. Reading file...");

    const fileData = await fs.readFile(filePath, 'utf-8');

    console.log("2. Parsing data...");

    const users = JSON.parse(fileData);

    console.log("3. Adding new user...");

    const newUser = {
      id: 3,
      name: "Usman",
      role: "Moderator"
    };

    users.push(newUser);

    console.log("4. Writing back to file...");

    await fs.writeFile(
      filePath,
      JSON.stringify(users, null, 2)
    );

    console.log("Success! Check users.json to see the new user.");

  } catch (error) {
    console.error("An error occurred:", error);
  }
}

manageUsers();
```

8. Working of the Program

The program follows these steps:

```
users.json
↓
Read file using fs.readFile()
↓
JSON text
↓
JSON.parse()
↓
JavaScript array
↓
Add new user
↓
JSON.stringify()
↓
Write using fs.writeFile()
↓
Updated users.json
```

The new user added during the practical was:

```
ID: 3
Name: Usman
Role: Moderator
```

10. Results

```
{
  "id": 1,
  "name": "Ali",
  "role": "Admin"
},
{
  "id": 2,
  "name": "Sara",
  "role": "User"
},
{
  "id": 3,
  "name": "Usman",
  "role": "Moderator"
}
]
```